

TaskFlow

REST API Backend Challenge

Career Fair • 2-Day Challenge

Duration	2 Days
Difficulty	Beginner → Intermediate
Language	Any — Python, Java, .NET, Node.js, Go, or anything you know
AI Allowed	Yes — use any AI tool, code generator, or assistant you like
Deliverable	A working REST API for a personal to-do list, testable in Postman
Frontend	Not required — bonus points for a UI, but the API is what counts
Docker	Required — your app, database, and dependencies must run via Docker Compose

Overview

TaskFlow is a personal to-do list API. A user should be able to create tasks, read them back, update them, and delete them — the four fundamental operations every backend developer must know: Create, Read, Update, Delete (CRUD).

Your job is to build a REST API that a client like Postman can call to manage a list of tasks. No frontend is required and no authentication is needed — all endpoints are open. Your data can live in memory, a file, or a database. The choice is yours.

This challenge is about backend fundamentals: clean API design, correct HTTP conventions, meaningful status codes, and solid input validation. Your entire stack — app, database, and dependencies — must run with a single Docker Compose command.

The Task Data Model

Every task in the system must have at minimum the following fields:

Task Object

```
id    string / int — Unique identifier — auto-generated by the server (required)
title string — Short description of the task (required)
description string — Longer detail about the task (optional)
status string — One of: pending, in_progress, completed (required)
priority string — One of: low, medium, high (required)
due_date string — ISO 8601 date string e.g. 2025-04-20 (optional)
created_at string — Timestamp set automatically when the task is created (required)
updated_at string — Timestamp updated automatically on every change (required)
```

Docker Requirements

Your entire project must be containerized and run with a single command. Judges will not install any dependencies manually if it does not run with Docker Compose, it will not be tested.

What You Must Provide

- A Dockerfile that builds your API server image
- A docker-compose.yml that wires together your app and database containers
- A .env or .env.example file for any environment variables (database URL, port, etc.)
- A README with one command to start everything: `docker compose up --build`

Required Endpoints

All endpoints must follow REST conventions. Use JSON for all request and response bodies.

Method	Endpoint	Description	Response
GET	/tasks	Return all tasks	200 — array of task objects
GET	/tasks/{id}	Return a single task by ID	200 task object, 404 if not found
POST	/tasks	Create a new task	201 — the created task object
PUT	/tasks/{id}	Replace a task entirely	200 updated task, 404 if not found
PATCH	/tasks/{id}	Update specific fields of a task	200 updated task, 404 if not found
DELETE	/tasks/{id}	Delete a task by ID	204 No Content, 404 if not found

Challenge Levels

Each level builds on the previous one. Judges will test all endpoints live in Postman.

LEVEL 1 - Core CRUD

Requirements

- Implement all 6 endpoints listed above
- Store tasks in memory (a list or dictionary in your code is fine)
- Auto-generate a unique ID and timestamps on task creation
- Return correct HTTP status codes for every response
- Return a clear error message and 404 when a task ID does not exist

Technical Hints

- *Set up your Dockerfile and docker-compose.yml first — get docker compose up --build running before writing any API code*
- *Start with GET /tasks and POST /tasks — get those working first before the rest*
- *For IDs: a simple auto-incrementing integer or UUID both work fine*
- *In Python: FastAPI or Flask; in Node: Express; in Java: Spring Boot; in .NET: minimal API or Web API*
- *At Level 1 store tasks in a module-level list in memory — no database container needed yet*
- *Test every endpoint in Postman before moving to the next level*

LEVEL 2 - Validation & Filtering

Requirements

- Validate all incoming request bodies — return 400 with a descriptive error if required fields are missing or invalid
- Reject invalid status values (anything other than pending, in_progress, completed)

Technical Hints

- *Validation libraries save time: Pydantic in Python, Joi in Node, Data Annotations in .NET*

- | | |
|---|---|
| <ul style="list-style-type: none"> • Reject invalid priority values (anything other than low, medium, high) • Add query parameter filtering to GET /tasks: ?status=completed, ?priority=high, ?due_date=2025-04-20 • Add sorting to GET /tasks: ?sort_by=due_date or ?sort_by=priority | <ul style="list-style-type: none"> • <i>For filtering: loop through the in-memory list and apply conditions before returning</i> • <i>Multiple filters should work together — ?status=pending&priority=high should AND the conditions</i> • <i>For sorting by priority, map the values to numbers (low=1, medium=2, high=3) before sorting</i> • <i>Return an empty array (not a 404) when filters match no tasks</i> |
|---|---|

LEVEL 3 - Persistence + Polish

Requirements

Technical Hints

- | | |
|--|--|
| <ul style="list-style-type: none"> • Persist tasks to a real storage layer — a database (SQLite, PostgreSQL, MySQL) or a JSON file • Tasks must survive a server restart • Add a GET /tasks/stats endpoint that returns a summary: total tasks, count by status, count by priority • Add a PATCH /tasks/{id}/complete shortcut endpoint that marks a task as completed in one call • Bonus: build a minimal UI — a simple HTML page or any frontend that calls your API | <ul style="list-style-type: none"> • <i>Add the db service to your docker-compose.yml — use postgres:15, mysql:8, or mongo:7</i> • <i>Connect your app container to the db using the service name as the hostname (e.g. host=db in your connection string)</i> • <i>Use a named volume in docker-compose.yml so database data persists across docker compose down / up cycles</i> • <i>The /stats endpoint is just a GET that aggregates your data — no body needed</i> • <i>The /complete shortcut is a PATCH that ignores the body and just sets status to completed</i> • <i>For the UI bonus: a plain HTML file with fetch() calls to your API is completely valid</i> |
|--|--|

Postman Testing Guide

Judges will use Postman to test your API live. Make sure your server is running and accessible on localhost before the demo. Here is what judges will test:

Sequence judges will run:

- POST /tasks — create 3 tasks with different statuses and priorities
- GET /tasks — verify all 3 are returned
- GET /tasks/{id} — fetch one by ID
- PATCH /tasks/{id} — update the status of one task
- PUT /tasks/{id} — replace a task entirely with new data
- GET /tasks?status=pending — verify filtering works
- DELETE /tasks/{id} — delete one task, verify GET /tasks no longer includes it
- GET /tasks/{bad-id} — verify a 404 is returned
- POST /tasks with missing title — verify a 400 is returned with an error message

Tips for a smooth demo:

- Export your Postman collection and have it ready to share with judges
- Make sure your server starts with a single command document it in your README
- Pre-load 2-3 sample tasks on startup so the GET /tasks response is not empty
- Check that your Content-Type header is application/json on all responses

Judging Criteria

Maximum score: 100 points. Judges will call all endpoints live in Postman during the demo.

Criteria	What We Look For	Points
CRUD Correctness	All 6 endpoints work and return correct data	30 pts
HTTP Conventions	Correct status codes, JSON responses, proper REST design	15 pts
Docker Setup	App and DB run correctly with docker compose up --build	25 pts
Validation	400s returned with clear messages for bad input	15 pts
Filtering & Sorting	Query parameters correctly filter and sort results	15 pts

- Extra 15 points for a good UI

Tips for Success

- Pick a framework you are comfortable with — this is not the time to learn a brand new language from scratch.
- Set up Docker first, before any API code — a containerized broken app is still better than a working app that only runs on your machine.
- Test in Postman after every single endpoint, not all at once at the end.
- HTTP status codes matter: 200 OK, 201 Created, 204 No Content, 400 Bad Request, 404 Not Found.
- A clean, well-validated Level 2 API beats a broken Level 3 every time.
- Write a README with your start command — judges should not have to figure out how to run your app.
- Use AI freely for boilerplate, debugging, or learning a framework quickly.